

Improved Silent Data Error Detection through Test Optimization using Reinforcement Learning

Manu Shamsa, John D. Martin, Mariano Phielipp, Thiago Macieira, Loganathan Lingappan, Brad Kelly, David Lerner, Michael Tucknott and Ethan Hansen

Intel Corporation

Abstract— Silent Data Errors (SDE) are a serious concern for at-scale computing in data centers because they may cause data loss or data corruption. It is therefore critical to identify defective devices that may experience SDE events and remove them before customer workloads are impacted. Because of the subtle nature of these defects, and the specific combination of events required for SDE to occur, detection is time consuming and costly. This paper reports the results of applying reinforcement learning methods to a family of open-source Eigen tests, and how this work improved the effectiveness of these tests at detecting SDE events.

Keywords: *Artificial Intelligence, Data Center Quality, Silent Data Error, Microprocessor Errors, Transistor Defects, System Reliability, Reinforcement Learning, Machine Learning.*

I. INTRODUCTION

Silent Data Errors (SDEs) present a significant challenge for the industry because of their ability to cause data loss or data corruption [1,2]. As node density in data centers increases exponentially, driven by at-scale compute needs for AI and cloud computing applications, the need to identify and remove defective devices that may cause SDE increases in importance. SDE are not unique to specific applications and have been observed to occur across compute architectures, process technologies, and vendors [3]. SDE primarily originate from manufacturing defects, which can be subtle, difficult to detect, and marginal in nature [4-5]. Many of these defects are latent and do not induce faults until after weeks, months, or even years of device use [6]. These characteristics are a challenge for standard scan-test methodologies that screen for SDE during manufacturing. Undetected defects that cause SDE may impact AI training or inference workloads, as well as general data center operations [1], [7]. Efficient SDE detection during both manufacturing test and periodic *in-situ* data center screening is essential for uninterrupted and reliable at-scale compute.

SDE occur even in System-on-Chip (SoC) regions with a high degree of structural or functional test coverage as measured by existing fault models. Defects located in the tail of the coverage saturation curve can escape and lead to SDE events [8]. To capture such defects, functional tests, such as those included in the Intel® Data Center Diagnostics Tool (DCDIAG) [7], [9-10] are required to exercise specific circuit paths to observe the marginal behavior and detect SDEs. The

SDE detection efficacy of DCDIAG and similar tools is determined by the testing periodicity and the specific tests run. Other factors that influence test efficacy include the number of nodes in the system, the components' predicted failure rate, and the application's sensitivity to SDE. In this paper we report results from our work to improve the SDE detection of tests by applying Reinforcement Learning (RL) to open-source tests that use the Eigen Linear Algebra library to achieve high SDE detection rate at lower test time, which is critical to manage the total cost of ownership (TCO) in data centers.

II. METHOD

The SDE detection tests used for this work are part of the Intel® Data Center Diagnostics Tool (DCDIAG), a tool developed by Intel to test Intel® Xeon® Processors for SDEs at manufacturing and in-field. The core of this tool and some of the tests present in it are available as the OpenDCDiag [10] open-source project. The full suite comprises of tests designed to verify that every operation and computation returns the correct result. These tests include checking core to core and socket to socket communications; exercising the various on-die caches; and executing nearly all floating point, integer, and data manipulation instructions. In some cases, those tests execute identical calculations across multiple cores in parallel and compare the results. Other tests perform reversible transformations, such as encryption/decryption or compression/decompression operations to identify data errors. To further expand the search space, the tests utilize pseudo-random number generation techniques provided in OpenDCDiag to span the vast data addresses and instruction space of a modern SoC device. The test suite provides broad coverage of most of the SoC functionality and can detect faults in the data paths, address paths, control logic, memory management, and execution units. Using these methods, Intel has successfully created tests capable of detecting rare defects that cause faults that manifest as SDE events.

The tests discussed in this paper are from the "Eigen" family of tests, which are available as open source in OpenDCDiag. These tests perform a variety of operations using matrices that are populated with random data to start. The Eigen tests are executed repeatedly for a specified duration, with different random data used for each iteration of the test. In the standard configuration, without RL guidance, the detection rate is low, and many repetitions are required to reliably detect SDE events [9]. In the AI-based Eigen test, the RL algorithm interacts with

Test Group	Test Name(s)	Matrix Sizes	Data Types
General Matrix Multiply	eigen_gemm_*	221x221 256x256	Double-precision complex Single- & double-precision real
Linear matrix equation solving	eigen_sparse	256x256	Double-precision real
Singular Value Decomposition using Bidiagonalization Divide & Conquer	eigen_svd_*	256x256 300x300 512x512	Single- & double-precision real Double-precision complex Single-precision real
Singular Value Decomposition using the Jacobi method	eigen_svd_jacobi_*	128x128 256x256 512x512	Double-precision complex Single- & double-precision real Single-precision real

Table I. List of tests in the Eigen family

the environment while we observe the current and historical details of the testing (defined as state) and choose an action (seed, value selection), following a policy, and then collect the reward (failure rate) at each step.

A. Eigen test in Sandstone for Matrix Multiplication

The Eigen tests were originally developed by Intel in 2019 to search for defects that other testing mechanisms had been unable to find. At the time, Intel had been in contact with customers who reported that certain workloads were suspected to experience SDE. In response, Intel created new tests to emulate the workloads found in Cloud Service Provider (CSP) data centers. The Eigen tests are intended to mimic large-scale floating-point data processing, which are common in AI workloads and other applications. The team was familiar with the Eigen Linear Algebra C++ template library [11], which allowed for rapid prototyping of new tests, as well as modifications to support different data types (single-precision vs double-precision, real vs complex numbers, etc.).

In this work we chose to target matrix multiplications specifically because of their heavy use of the Fused-Multiply-Add (FMA) instructions that are used to optimize mathematical operations. The Intel performance cores used in Xeon processors can complete up to 8 double-precision, 16 single-precision, or 32 half-precision multiply-and-add operations in parallel in just 4 cycles. These cores dedicate significant silicon area to implementing the FMA instructions. Because large functional areas will generally experience more faults caused by randomly distributed manufacturing defects, the FMA region was a good choice for test optimization.

In addition, because of their size and relatively high-power consumption, the Intel processor FMA units are powered down when not in use, to reduce the overall power requirements of the SoC. Consequently, tests that make heavy use of matrix multiplication will cause repeated power-up/power-down of the FMA, creating opportunities for voltage and current transients to activate marginal defects which could manifest as SDE, and

that may be undetected by stuck-at test methods. In addition, these tests may induce analog signal imperfections, including low intermediate values, spikes, or high first-order derivatives of the levels. In addition, changes in current produce further electromagnetic effects that could affect other IP blocks on the chip that may not be directly electrically connected.

Based on 5 years of data collected from running these tests in manufacturing, we have observed that while the Eigen tests are effective at finding defects that can manifest as SDE, the search space is vast due to the many large data types and broad instruction set. This leads to a high average time-to-failure and relatively poor test efficiency.

The Eigen family of tests is written in C++ and, like the Eigen Linear Algebra library that they depend on, make heavy use of templates. The OpenDCDiag Git repository contains four different types of tests, as described in Table I. They all follow the same testing procedure, illustrated by Fig. 1, in which the computation is checked against an expected result calculated by one of the CPU cores under test. This paper focuses on the tests that perform a General Matrix Multiplication (GEMM), found in the `tests/eigen_gemm` subdirectory in the source code.

B. Motivation to Fine-tune Input Parameters of Eigen Test

Eigen algorithms rely on different input parameters, with matrix size being a key factor. In our study, we investigate how matrix size impacts the utilization of various components within the processor micro-architecture. To achieve this, we follow these steps:

1. Extract instruction traces from individual runs of an GEMM algorithm implemented by the Eigen library, varying the matrix size.
2. Feed these traces into a micro-architectural simulator.
3. Analyze metrics related to component utilization.

Fig. 2 illustrates the utilization of different micro-architectural areas as a percentage of the maximum observed

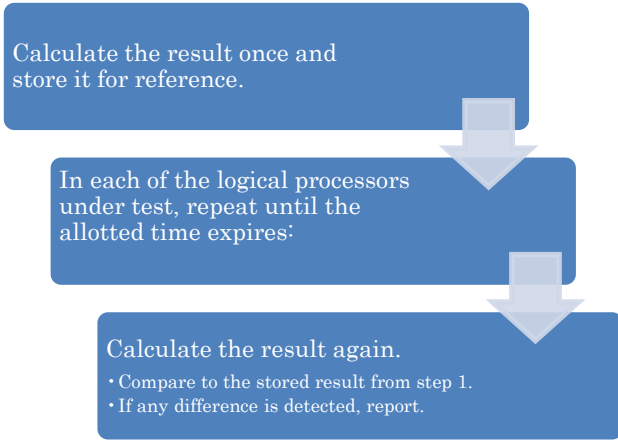


Fig. 1. Eigen test algorithm for SDE detection

utilization across various matrix sizes. Notably, as matrix dimensions increase, workloads shift toward memory-bound and execution unit-bound tasks, spending less time in the front-end of the microarchitecture. The impact of matrix dimensions on out-of-order utilization appears weaker. Additionally, the profile captures distinct algorithm behaviors: for small matrix sizes, the algorithm avoids dynamic memory allocation and favors loop unrolling. Conversely, for large matrices, cache handling receives special attention to optimize performance.

Fig. 3-5 provide a breakdown of utilization within execution, memory, and out-of-order clusters, respectively. In the execution cluster, increasing matrix sizes lead to higher average utilization of FMA and SIMD (single instruction multiple data) units but reduce the utilization of integer execution units. Meanwhile, in the memory cluster (Fig. 4), the impact of matrix sizes on memory-order unit utilization is weak beyond the 32x32 size. The same trend holds for the regions shown in Fig. 5 within the Out-Of-Order cluster. Figures 2-5 underscore the importance of selecting appropriate matrix sizes to induce specific stress in the processor. For tasks requiring significant FMA activity, larger matrix sizes are preferable. However, it's essential to recognize that computations with large matrices take more time, resulting in extended run/test durations per iteration. To target areas like translation lookaside buffer (TLB), memory-order, and out-of-order regions, it is worthwhile to run more iterations with smaller matrix sizes (e.g., 32x32 or 64x64).

Note that this analysis is for a specific version of `eigen_gemm` test. Profiles may vary based on the underlying algorithm.

C. Reinforcement Learning Application to the selection of OpenDCDiag Inputs

The number of possible inputs to an Eigen test is too large for an exhaustive evaluation to be feasible. Current methods typically adopt a randomized approach that evaluate inputs with uniform sampling. However, for error detection, this approach is inefficient, as there is no principle to guide the selection of inputs. In this work we investigate the application of

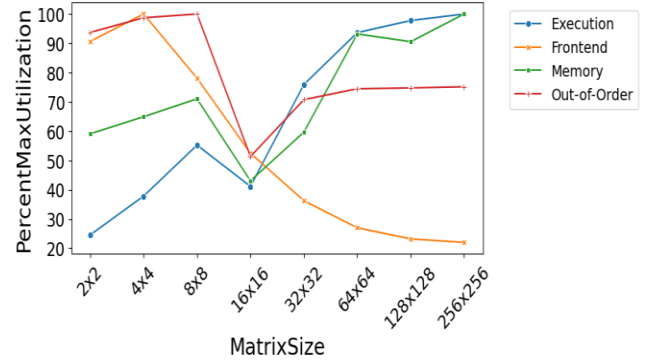


Fig. 2. Utilization profile of core micro-architectural regions for different matrix sizes

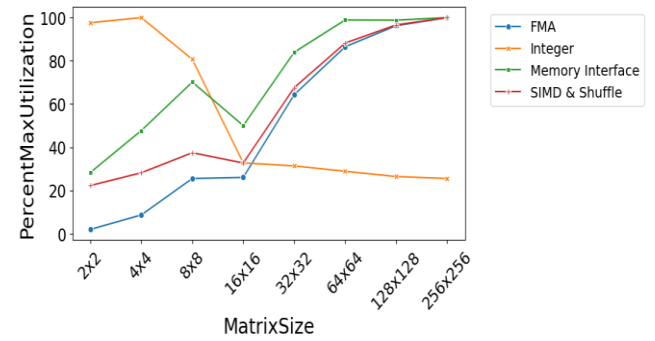


Fig. 3. Utilization profile breakdown within Execution cluster for different matrix sizes

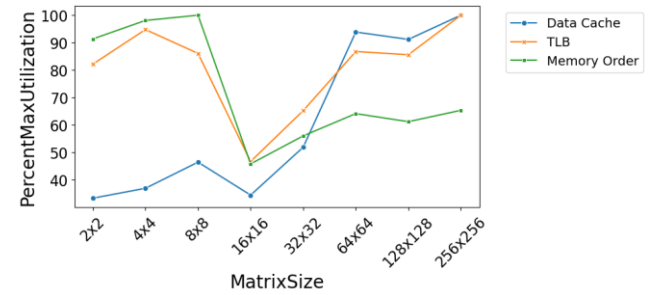


Fig. 4. Utilization profile breakdown within Memory cluster for different matrix sizes

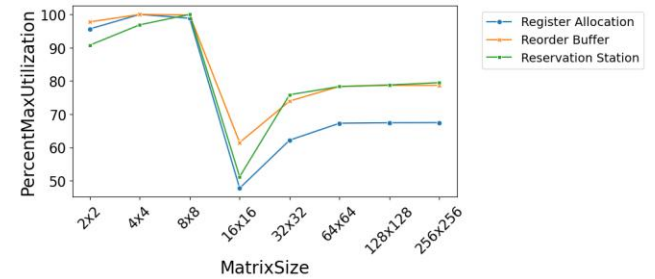


Fig. 5. Utilization profile breakdown within Out-Of-Order cluster for different matrix sizes

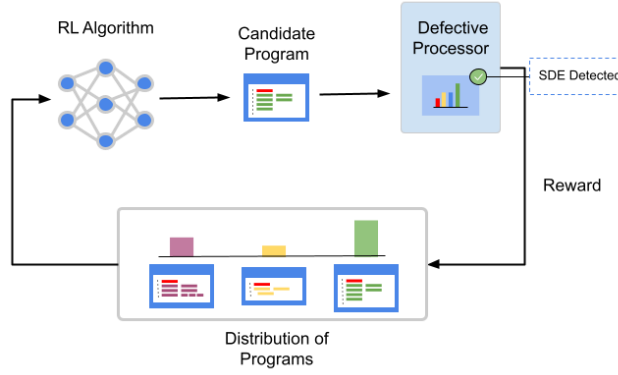


Fig. 6. Architecture of RL system applied to detect SDEs with Eigen programs.

reinforcement learning to systematically explore the space of inputs and learn a set of test inputs that maximizes the error detection rate.

Fig. 6 shows the RL architecture we implemented for the Eigen tests. Reinforcement Learning is a computational model of trial-and-error learning [12]. Within its framework, problems are viewed as an ongoing interaction between an agent and an environment. Where at every moment in time, the agent takes an action based on some observable data and receives, in response, a new observation along with a reward signal indicating the optimality of the previous action. Our work views the environment as a test program executing on a potentially defective processor. The agent is a learning algorithm that selects different test inputs at each step. Based on the feedback in the reward, which indicates failure, the learning algorithm will update its decision policy and over time learn to select test inputs that maximize the detection rate.

Our study considered tests based on standard matrix multiplication, where the content of each matrix was parameterized by a single number from a finite set. The agent represented a discrete distribution over these possibilities and updated the probabilities using the EXP3 learning algorithm [13].

Our first experiment examined the performance of the learning algorithm compared to two baselines. The first baseline used a naive strategy that uniformly samples from the set of possible inputs. The inputs are parameterized by a “matrix generator”: a seed that determines the matrix elements. We consider matrix generators with values from 1,2,...,9. The second baseline used an arbitrary constant strategy: always choosing the same input. The randomized approach is intended to reflect the performance of current methods based on the same sampling strategy over a much larger input space. And the constant baseline, which was also characterized for all possible inputs, is used as another method that provides a second

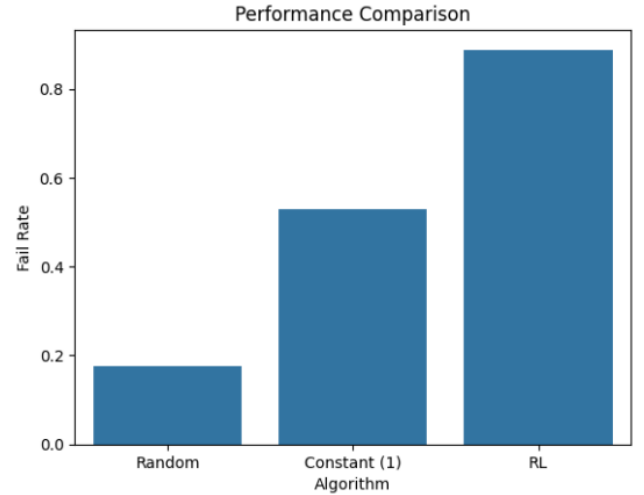


Fig. 7. Summary of automated SDE detection performance.

baseline to compare against. Learning occurred over one-thousand interactions.

Our second set of results aims to characterize the trend in failure rate as the size of test program increases. We consider several different values that use an increasing amount of memory.

III. RESULTS AND DISCUSSION

The results in Fig. 7 demonstrate that, for this defect, a naive random strategy is least likely to result in a detection. The RL-based method identifies the program input achieving the highest failure rate, and it selects that repeatedly. Fig. 8 illustrates the evolution of the agent’s policy, which is a distribution over Eigen test inputs. As one can see, the probability of selecting the most effective inputs (induced by the matrix generator “7”) stabilizes sometime before the full 1000 iterations complete. Note the probability of selecting an input is different than the failure rate. These results provide evidence showing there are cases where RL-based methods are effective at detecting SDEs. Furthermore, these results demonstrate detection can be achieved with a reasonable amount of data and resources.

These results indicate that as more memory is used for the test, the chance of detecting an SDE increases, likely due to increased functional block utilization as discussed previously in section II.B. The system-based test [14] used by Intel to manufacture current Xeon® processors is now utilizing Eigen tests that have been optimized using RL-based method. Future work will assess the effectiveness of the learning algorithm on additional products, to understand whether additional tuning is needed for new designs. The effectiveness of the RL optimized tests will also be evaluated for in-field testing in the future.

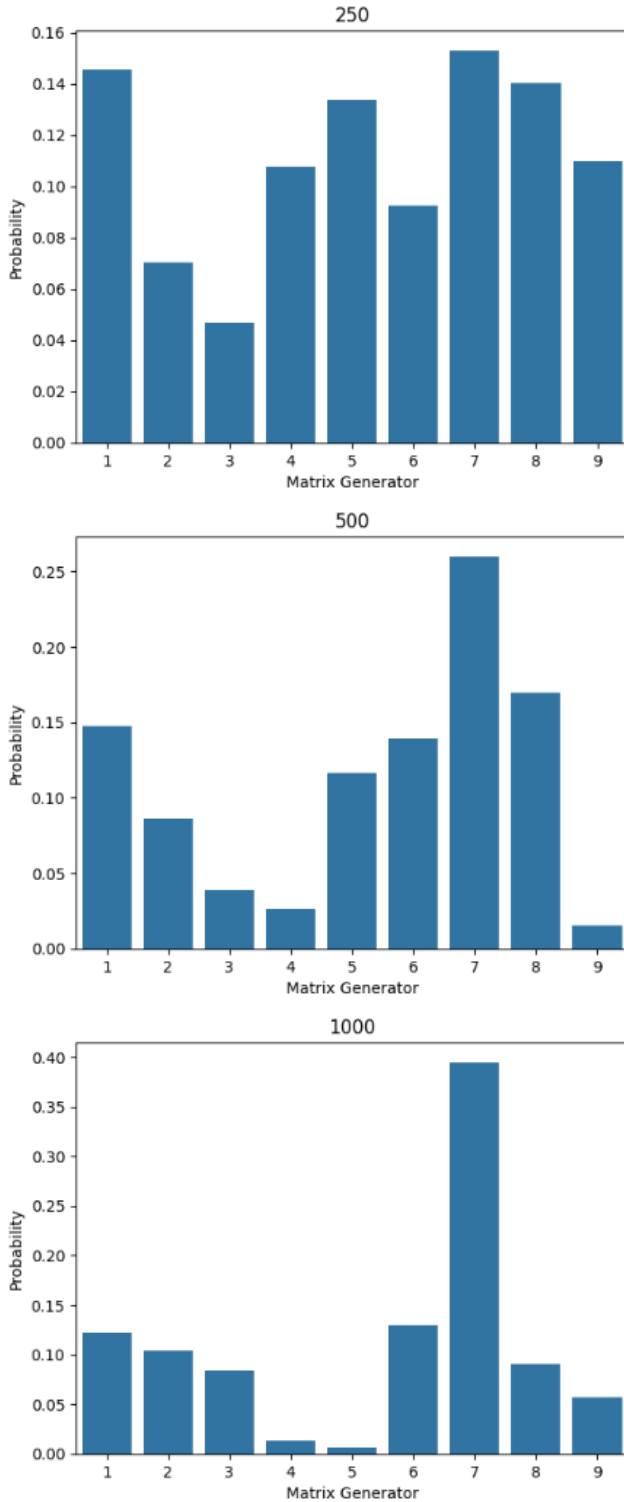


Fig. 8. Evolution of Distribution Over Matrix Generators (Actions): at 250, 500, and 1000 learning iterations.

IV. CONCLUSION

SDE presents a significant challenge for the industry due to their high impact on critical workloads and datacenter operations. Efficient detection of SDE is critical for reliable at-scale compute in datacenters and the TCO. Our results demonstrate that OpenDCDiag provides a framework to modify open-source tests such as the Eigen family of tests. By using advanced AI algorithms to optimize the test parameters, more efficient detection of SDE may be achieved without an increase in test time and cost.

ACKNOWLEDGMENT

The authors gratefully acknowledge the contributions of Balkaran Gill, David Pullen, Somdeb Majumdar and Brad Bittel.

REFERENCES

- [1] D. Lerner, M. Shamsa, B. Bittel and B. Inkley, "Silent Data Errors: Causes, Implications to Artificial Intelligence, & Mitigation," *IEEE RAS in Data Centers Summit 2024*.
- [2] H. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, "Silent data corruptions at scale," *CoRR*, vol. abs/2102.11245, 2021.
- [3] P. Hochschild, P. Turner, J. Mogul, R. Govindaraju, P. Ranganathan, D. Culler, and A. Vahdat, "Cores that don't count," in *Proc. 18th Works. on Hot Topics in Oper. Sys. (HotOS 2021)*, 2021.
- [4] M. Shamsa and D. Lerner, "Defect Mechanisms Responsible for Silent Data Errors," *IEEE Int. Reliab. Physics. Symp.*, 2024.
- [5] P. G. Ryan, I. Aziz, W. B. Howell, T. K. Janczak and D. J. Lu, "Process defect trends and strategic test gaps," *IEEE Int. Test Conf.*, 2014, pp. 1.
- [6] S. Natarajan, A. Sathaye, C. Oak, N. Chaplot and S. Banerjee, "DEFCON: Defect Acceleration through Content Optimization," 2022 IEEE International Test Conference (ITC), Anaheim, CA, USA, 2022, pp. 298-304, doi: 10.1109/ITC50671.2022.00038.
- [7] B. Bittel, M. Shamsa, B. Inkley, A. Gur, D. Lerner, and M. Adams, "Data Center Silent Data Errors: Implications to Artificial Intelligence Workloads & Mitigations," *IEEE Int. Reliab. Physics. Symp.*, 2024.
- [8] W. Li, C. Nigh, D. Duvalsaint, S. Mitra and R. D. Blanton, "PEPR: Pseudo-Exhaustive Physically-Aware Region Testing," 2022 IEEE International Test Conference (ITC), Anaheim, CA, USA, 2022, pp. 314-323, doi: 10.1109/ITC50671.2022.00083.
- [9] D. P. Lerner, B. Inkley, S. H. Sahasrabudhe, E. Hansen, L. D. R. Munoz and A. v. de Ven, "Optimization of Tests for Managing Silicon Defects in Data Centers," 2022 IEEE International Test Conference (ITC), Anaheim, CA, USA, 2022, pp. 578-582, doi: 10.1109/ITC50671.2022.00076.
- [10] A. van de Ven and I. Steiner, "A Software Solution for Managing Silicon Defects in Data Centers," *IEEE Int. Test Conf.*, 2021.
- [11] B. Jacob, "Eigen is a C++ template library for linear algebra," Eigen, <https://eigen.tuxfamily.org/> (accessed Apr. 25, 2024).
- [12] R. S. Sutton, and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [13] Y. Seldin, C. Szepesvári, P. Auer, Y. Abbasi-Yadkori, "Evaluation and Analysis of the Performance of the EXP3 Algorithm in Stochastic Environments," *JMLR: Workshop and Conference Proceedings 24: 103-116*, 2012 10th European Workshop on Reinforcement Learning
- [14] S. Biswas and B. Cory, "An industrial study of system-level test," *IEEE Design & Test of Computers*, vol. 29 (1), pp. 19-27, 2012.